NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 09

AIM :- To design and implement a Long Short-Term Memory (LSTM) model for sequence learning and time series forecasting.

OBJECTIVE :-

1. To understand the concept of Long Short-Term Memory networks.
2. To understand sequence learning using LSTM.
3. To understand the role of memory cells and gates in LSTM.
4. To design an LSTM model for time series forecasting.
5. To train the LSTM model using sequential data.
6. To visualize the actual and predicted values.

THEORY :-

1. Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is a special type of Recurrent Neural Network (RNN) designed to process sequential data and learn dependencies over time. Traditional RNNs can have difficulty remembering information from earlier time steps because of the vanishing gradient problem. LSTM overcomes this limitation using a memory cell and a system of gates that control the flow of information. LSTMs are widely used for time series forecasting, text processing, speech recognition and other sequence-based tasks.

In time series forecasting, observations are arranged according to their order in time. The previous values of a sequence can contain useful information for predicting a future value. LSTM is suitable for this task because it can learn patterns and dependencies from previous time steps.

2. Sequential Data

Sequential data is data in which the order of observations is important. Examples include temperature measurements, stock prices, sales, weather observations, speech and text. For example, a simple time series can be represented as:

$$X = [x1, x2, x3, x4, \ldots, xt]$$

where each value represents an observation at a particular time. For forecasting, a sequence of previous values can be used to predict the next value. For example, if three previous values are used as input:

$$[x1, x2, x3] \rightarrow x4$$

The LSTM learns this relationship from many such sequences during training.

3. LSTM Architecture

An LSTM unit contains a cell state and a hidden state. The cell state carries important information through the sequence, while the hidden state represents the output of the LSTM at a particular time step.

The main components of an LSTM are the forget gate, input gate and output gate. These gates control which information should be forgotten, stored and passed to the next time step.

4. Forget Gate

The forget gate determines which information from the previous cell state should be retained or discarded. It receives the current input xt and previous hidden state ht−1 and produces values between 0 and 1 using the sigmoid function.

$$ft = \sigma(Wf[ht-1, xt] + bf)$$

A value close to 1 indicates that the information should be retained, while a value close to 0 indicates that it should be forgotten.

5. Input Gate

The input gate determines which new information should be added to the cell state. It uses the current input and previous hidden state to decide which information is important.

$$it = \sigma(Wi[ht-1, xt] + bi)$$

A candidate cell state is also generated to represent new information that may be stored.

6. Cell State

The cell state acts as the long-term memory of the LSTM. It is updated using information selected by the forget and input gates.

$$Ct = ft * Ct-1 + it * \tilde{C}t$$

This mechanism allows the LSTM to retain important information over multiple time steps.

7. Output Gate

The output gate determines which information from the updated cell state should be passed to the hidden state.

$$ot = \sigma(Wo[ht-1, xt] + bo)$$

The hidden state is then calculated using:

$$ht = ot * tanh(Ct)$$

The hidden state is passed to the next time step and can also be used to produce the final prediction.

8. LSTM for Time Series Forecasting

Time series forecasting involves predicting future values based on previously observed values. In an LSTM forecasting model, historical observations are converted into sequences that can be provided to the network. For example, using a sequence length of 10, the first ten observations are used to predict the eleventh observation:

$$[x1, x2, \ldots, x10] \rightarrow x11$$

The next sequence can then be:

$$[x2, x3, \ldots, x11] \rightarrow x12$$

This sliding-window approach converts the time series into input-output pairs suitable for supervised learning.

9. Data Preprocessing

Before training the LSTM, the time series values can be normalized to a smaller range. Normalization helps the neural network train more effectively. In this experiment, the sequence is scaled between 0 and 1 using:

$$x' = (x - x_min) / (x_max - x_min)$$

The scaled data is then divided into training sequences and target values.

10. LSTM Architecture Used

The LSTM model used in this experiment follows a stacked-gate architecture (forget, input, output gates around a memory cell) repeated across each time step. The LSTM layer processes the sequence and learns temporal relationships between the observations. The Dense layer produces the final numerical prediction.

11. Training the Model

The LSTM model is compiled using the Adam optimizer and Mean Squared Error (MSE) loss function. MSE is suitable for numerical forecasting because it measures the squared difference between the actual and predicted values.

$$MSE = (1/n) \Sigma (yi - \hat{y}i)^2$$

During training, the model adjusts its weights using backpropagation through time so that the prediction error is reduced. LSTM's gated structure helps maintain useful information across multiple time steps.

12. Working of LSTM Forecasting

The working process can be summarized as follows: generate or load the sequential time series data; normalize the data; create sequences using a fixed time window; use previous observations as input and the next observation as the target; provide the sequences to the LSTM layer; the LSTM processes the sequence using its memory cell and gates; the Dense layer produces the predicted value; calculate the prediction error using MSE; update the model weights during training; compare the actual and predicted values using a graph.

13. Applications of LSTM

LSTM networks are used in many sequence-learning applications. They can be used for time series forecasting, weather prediction, stock and demand forecasting, speech processing, language modelling and text prediction. Their ability to capture long-term dependencies makes them suitable for problems where previous observations influence future outputs.

PROCEDURE :-

1. Import TensorFlow, NumPy and Matplotlib.
2. Generate the sequential time series data.
3. Normalize the data.
4. Create input sequences and target values.
5. Split the sequences into training and testing data.
6. Create the LSTM model.
7. Add an LSTM layer with 50 units.
8. Add a Dense layer for numerical prediction.
9. Compile the model using Adam optimizer and MSE loss.
10. Train the model using the generated sequences.
11. Predict the values for the test sequences.
12. Visualize the actual and predicted values.

CODE :-

```
import numpy as np

import tensorflow as tf

import matplotlib.pyplot as plt


x = np.linspace(0, 100, 1000)

data = np.sin(x)


data_min = data.min()

data_max = data.max()


scaled_data = (data - data_min) / (data_max - data_min)


sequence_length = 20


X = []

y = []


for i in range(len(scaled_data) - sequence_length):

    X.append(scaled_data[i:i + sequence_length])

    y.append(scaled_data[i + sequence_length])
```

```
X = np.array(X)
y = np.array(y)

X = X.reshape((X.shape[0], X.shape[1], 1))

split = int(len(X) * 0.8)

X_train = X[:split]
y_train = y[:split]

X_test = X[split:]
y_test = y[split:]

model = tf.keras.Sequential([
    tf.keras.Input(shape=(sequence_length, 1)),
    tf.keras.layers.LSTM(50),
    tf.keras.layers.Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mse"
)

history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2
)

predictions = model.predict(X_test, verbose=0)

predictions = predictions * (data_max - data_min) + data_min
actual = y_test * (data_max - data_min) + data_min
```

```
test_loss = np.mean((actual - predictions.flatten()) ** 2)

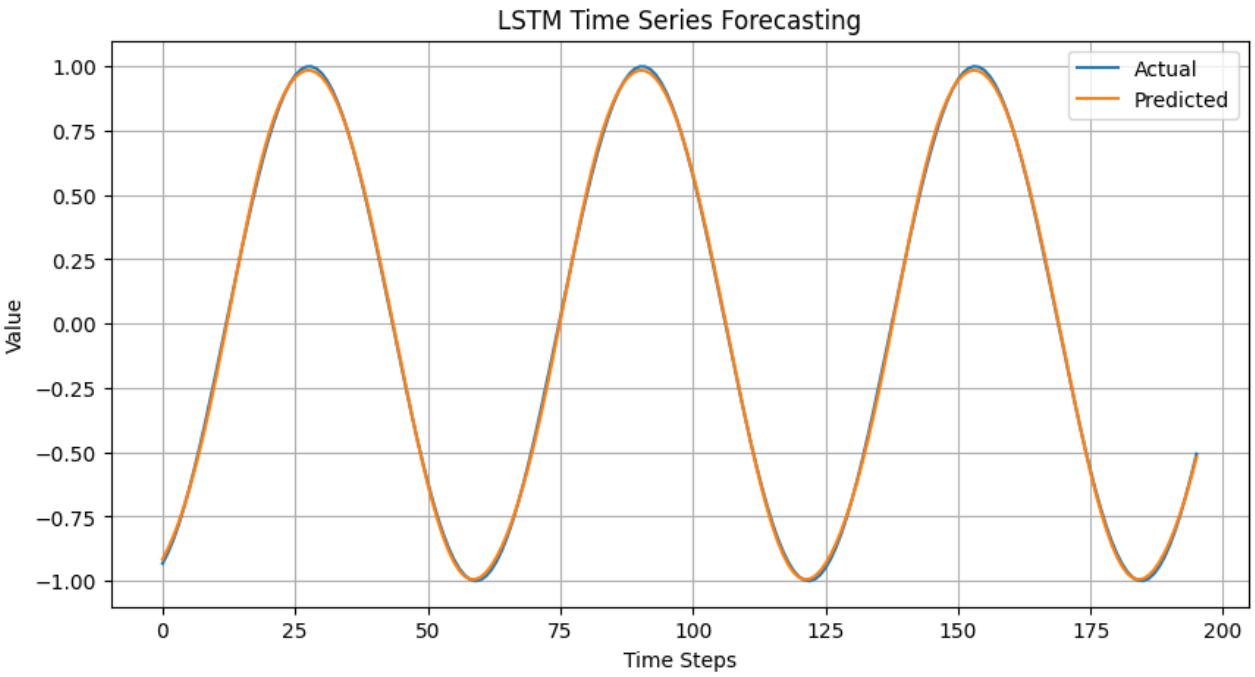

print("Test MSE:", test_loss)

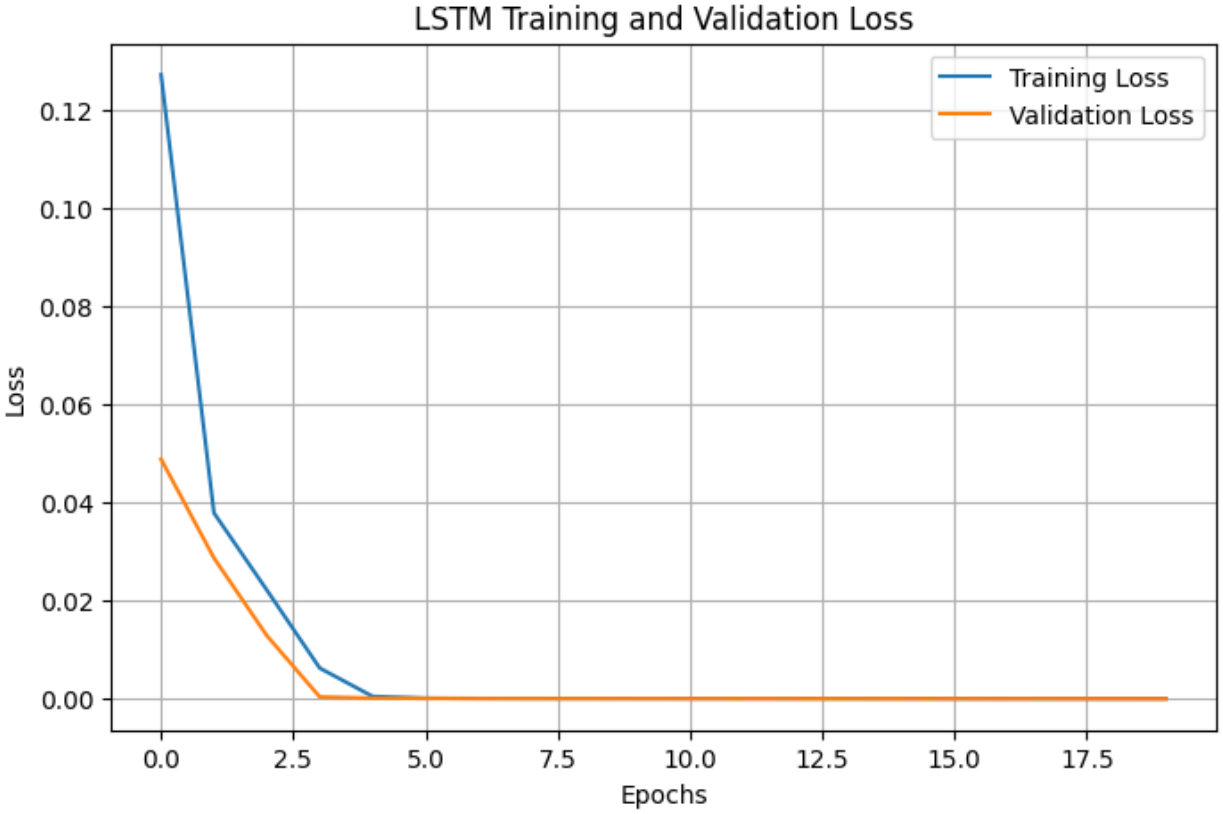

plt.figure(figsize=(10, 5))
plt.plot(actual, label="Actual")
plt.plot(predictions, label="Predicted")
plt.xlabel("Time Steps")
plt.ylabel("Value")
plt.title("LSTM Time Series Forecasting")
plt.legend()
plt.grid(True)
plt.show()


plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("LSTM Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT :-

```
Epoch 1/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 3s 79ms/step - loss: 0.1272 - val_loss: 0.0489
Epoch 2/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step - loss: 0.0379 - val_loss: 0.0287
Epoch 3/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0222 - val_loss: 0.0129
Epoch 4/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0063 - val_loss: 4.1462e-04
Epoch 5/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 1s 27ms/step - loss: 5.1719e-04 - val_loss: 1.5852e-04
Epoch 6/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.4657e-04 - val_loss: 9.9647e-05
Epoch 7/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 9.3025e-05 - val_loss: 8.1469e-05
Epoch 8/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.9961e-05 - val_loss: 7.3923e-05
Epoch 9/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.3976e-05 - val_loss: 6.9156e-05
Epoch 10/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 6.9469e-05 - val_loss: 6.4846e-05
Epoch 11/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 6.5232e-05 - val_loss: 6.0302e-05
Epoch 12/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 6.1120e-05 - val_loss: 5.5757e-05
Epoch 13/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 5.7123e-05 - val_loss: 5.1449e-05
Epoch 14/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 5.3258e-05 - val_loss: 4.7574e-05
Epoch 15/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 4.9573e-05 - val_loss: 4.4060e-05
Epoch 16/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 4.6078e-05 - val_loss: 4.0719e-05
Epoch 17/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 4.2767e-05 - val_loss: 3.7523e-05
Epoch 18/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 3.9629e-05 - val_loss: 3.4508e-05
Epoch 19/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.6663e-05 - val_loss: 3.1678e-05
Epoch 20/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.3873e-05 - val_loss: 2.9022e-05
```


LSTM Time Series Forecasting
Actual
Predicted
Value
Time Steps
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
25
50
75
100
125
150
175
200


LSTM Training and Validation Loss
Training Loss
Validation Loss
Loss
Epochs
0.12
0.10
0.08
0.06
0.04
0.02
0.00
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

CONCLUSION :-

The experiment successfully implemented an LSTM model for sequence learning and time series forecasting. The LSTM learned temporal patterns from previous observations and generated predictions for future values. The actual and predicted values were visualized to evaluate the forecasting performance of the model.